Lab8

Data Structure

Lab8

- The deadline for lab8 submission is 30th April at 11:59 pm.
- If you have any questions, please contact TA through email. (justin043435@hanyang.ac.kr)
- Lab scores will be uploaded to the LMS next week.
- Questions about Lab 7 scores will only be accepted during next lab session.

- Code name: {student_id}_p8.c
- The code will be tested with 5 different input files.
- Each input file will be worth 20 points
- Make sure that the output format matches the provided example exactly.

Evaluation criteria

Category	Evaluation	
p8	100	
Total	100	

- *Use GCC **13.3** version.*
- *No score will be given if there is a compile error caused by difference of gcc version*

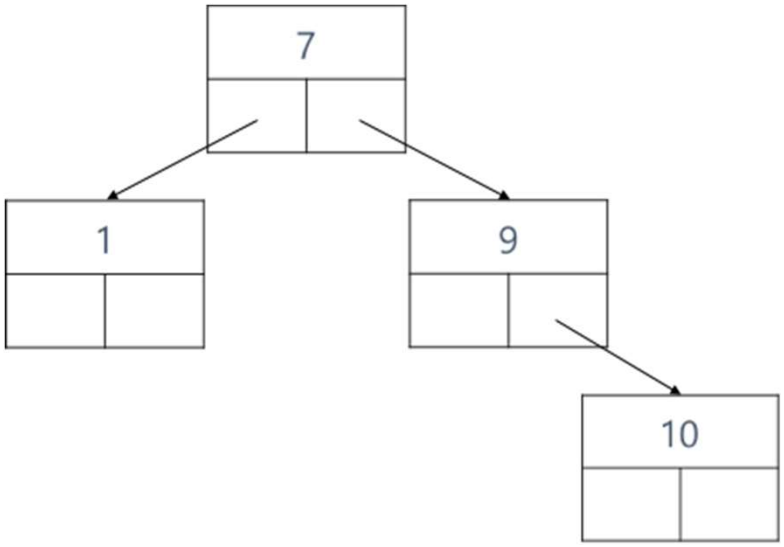
Lab 8. AVL Tree

- Design and implement an AVL Tree using a linked list structure.
- Ensure that the AVL Tree maintains balance through rotations (LL, RR, LR, RL).
- The input file is a sequence of integers. Insert each integer into the AVL Tree in the order it appears.
- After all integers are inserted, perform an in-order traversal and print each node with its height.
- Use **recursive functions** for:
  - Insertion
  - In-order traversal (used in the output)

Lab8. AVL Tree

- Structure

```
typedef struct AVLNode *AVLTree;
typedef int ElementType;

struct AVLNode
{
        ElementType Element;
        AVLTree Left;
        AVLTree Right;
        int Height;
};
```



Lab8. AVL Tree

- Structure

```
typedef struct AVLNode *Position;
typedef struct AVLNode *AVLTree;
typedef int ElementType;

struct AVLNode
{
        ElementType Element;
        AVLTree Left;
        AVLTree Right;
        int Height;
};
```

- Function

```
int Max(ElementType num1, ElementType num2);
int Height(Position P);
Position SingleRotateWithLeft(Position node);
Position SingleRotateWithRight(Position node);
Position DoubleRotateWithLeft(Position node);
Position DoubleRotateWithRight(Position node);
AVLTree Insert(ElementType X, AVLTree T);
void PrintInorder(AVLTree root);
void DeleteTree(AVLTree T);
```

Lab8. AVL Tree

- **int Max(ElementType num1, ElementType num2)**
  - Return the larger of two integers.
- **int Height(Position P)**
  - Return the height of a given node.
- **Position SingleRotateWithLeft(Position node)**
  - Perform single rotation with left (LL case).
- **Position SingleRotateWithRight(Position node)**
  - Perform single rotation with right (RR case).
- **Position DoubleRotateWithLeft(Position node)**
  - Perform double rotation with left (LR case).
- **Position DoubleRotateWithRight(Position node)**
  - Perform double rotation with right (RL case).

Lab8. AVL Tree

- **AVLTree Insert(ElementType X, AVLTree T)**
  - Insert a new node with the specified key "X" into the AVL tree.
  - If key 'X' is already in the tree, print an error message: "Insertion Error: [key] is already in the tree!".
- **void PrintInorder(AVLTree T)**
  - Print all nodes of the tree in in-order traversal.
- **void DeleteTree(AVLTree T)**
  - Free all nodes in the tree to release memory.

Lab8. AVL Tree – Output Message Format

- **Rotation Messages (on rebalancing):**
  - "There's node([key]) to be balanced! Do SingleRotateWithLeft!"
  - "There's node([key]) to be balanced! Do SingleRotateWithRight!"
  - "There's node([key]) to be balanced! Do DoubleRotateWithLeft!"
  - "There's node([key]) to be balanced! Do DoubleRotateWithRight!"

- **Inorder Traversal Output Format:**
  - "[key1](height1) [key2](height1) … "

Lab8. AVL Tree

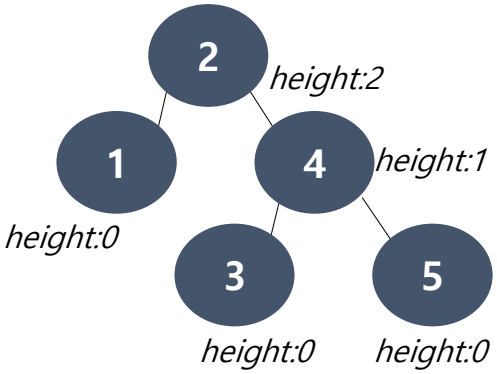
```c
#include <stdio.h>
#include <stdlib.h>

struct AVLNode;
typedef struct AVLNode *Position;
typedef struct AVLNode *AVLTree;
typedef int ElementType;

struct AVLNode{
        ElementType Element;
        AVLTree Left;
        AVLTree Right;
        int Height;
};

int Max(ElementType num1, ElementType num2);
int Height(Position P);
Position SingleRotateWithLeft(Position P);
Position SingleRotateWithRight(Position P);
Position DoubleRotateWithLeft(Position P);
Position DoubleRotateWithRight(Position P);
AVLTree Insert(ElementType X, AVLTree T);
void PrintInorder(AVLTree T);
void DeleteTree(AVLTree T);
```

```c
int main(int argc, char *argv[]){

        AVLTree myTree = NULL;
        int key;

        FILE *fi = fopen(argv[1], "r");
        while (fscanf(fi, "%d", &key) != EOF){
                myTree = Insert(key, myTree);
        }
        fclose(fi);

        PrintInorder(myTree);
        printf("\n");

        DeleteTree(myTree);
        return 0;
}


void DeleteTree(AVLTree T){
        if (T->Left != NULL)
                DeleteTree(T->Left);
        if (T->Right != NULL)
                DeleteTree(T->Right);
        free(T);
}
```

Lab8. AVL Tree – Simple Example

- input file : lab8_input.txt

```
3 2 1 4 5
```



- Result

```
There's node(3) to be balanced! Do SingleRotateWithLeft!
There's node(3) to be balanced! Do SingleRotateWithRight!
1(0) 2(2) 3(0) 4(1) 5(0)
```

Lab8. AVL Tree – Simple Example

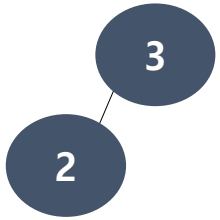
Input : 3 2 1 4 5

Output

Insert 3

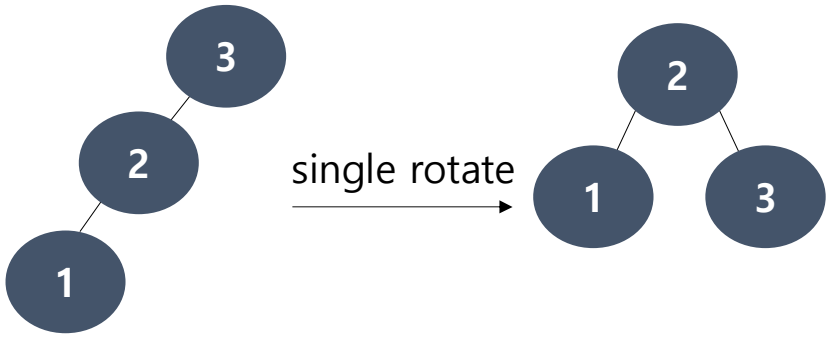
nothing

Insert 2



nothing

Insert 1



There's node(3) to be balanced! Do SingleRotateWithLeft!

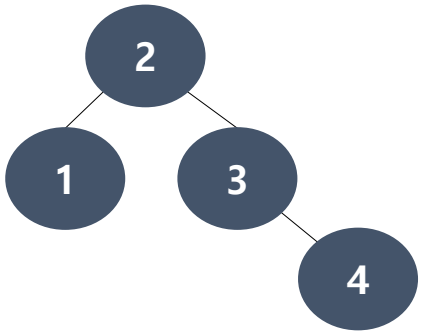
Lab8. AVL Tree – Simple Example

Input : 3 2 1 4 5

Output

nothing

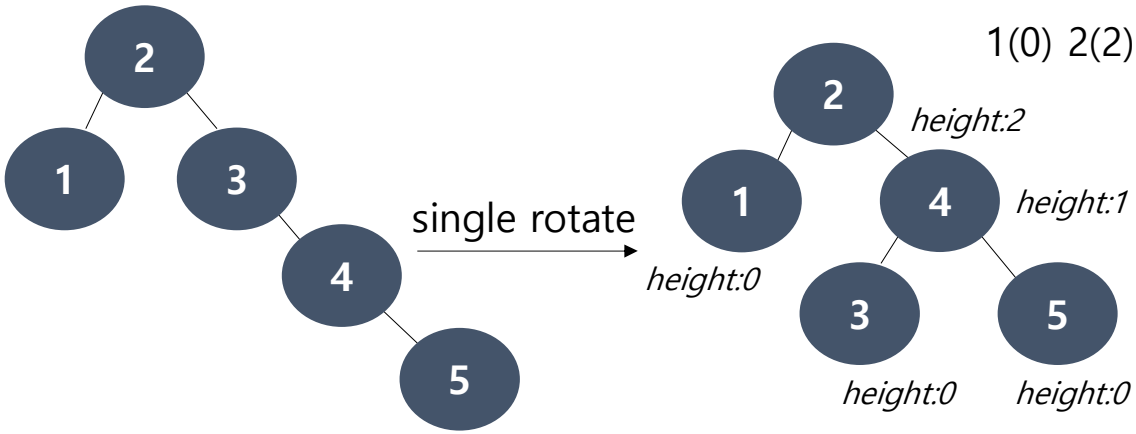
Insert 4



Insert 5

There's node(3) to be balanced! Do SingleRotateWithRight!

1(0) 2(2) 3(0) 4(1) 5(0)



Lab8. AVL Tree – Example

- input file : lab8_input2.txt

```
7 5 3 10 23 4 20 21 22 23 24 25
```

- Result

```
There's node(7) to be balanced! Do SingleRotateWithLeft!
There's node(7) to be balanced! Do SingleRotateWithRight!
There's node(23) to be balanced! Do DoubleRotateWithLeft!
There's node(10) to be balanced! Do SingleRotateWithRight!
Insertion Error: 23 is already in the tree!
There's node(5) to be balanced! Do SingleRotateWithRight!
3(1) 4(0) 5(2) 7(0) 10(1) 20(0) 21(3) 22(0) 23(2) 24(1) 25(0)
```